

Concurrency / Atomicity Test Strategy

Concurrency / Atomicity Test Strategy

Revision: 1 Last modified: 2026-06-26T12:00:00Z

Master technical specification — Volume 8 (Testing & QA), nano-detail document **concurrency**, one of the seven §11.4.169 cross-cutting test-type deep-dives. It deepens [10-testing-acceptance-and-qa.md](#) §5.10 (CONC) into an implementation-ready bank for correctness under concurrent callers, transactional **atomicity**, and **idempotency under replay**. SPEC-ONLY: it describes the harness, fixtures, evidence, gate, and the paired §1.1 mutation; it does not build the product. Sources cited inline by id — [OVERVIEW] = doc 10; [ipam] = [../v03-control-plane/svc-ipam.md]; [reconcile] = [../v03-control-plane/reconciliation-flow.md]; [svc-policy] = [../v03-control-plane/svc-policy.md]; [TM] = [../v05-security/threat-model.md]. Claims not grounded in the evidence base are marked **UNVERIFIED** per constitution §11.4.6 — never fabricated. (Companion: race/deadlock detection is owned by [race-deadlock.md]; this doc owns *logical* correctness under concurrency, that doc owns *memory-model* races.)

Table of contents

- 0. Scope on HelixVPN surfaces
- 1. The three correctness properties
- 2. Harness
- 3. Fixtures — real DB, no mocks (§11.4.27)
- 4. Captured evidence (§11.4.69)
- 5. Determinism (§11.4.50)
- 6. Acceptance gate

- [7. The paired §1.1 mutation](#)
- [8. Test skeletons](#)
- [Sources verified](#)

0. Scope on HelixVPN surfaces

Project principle 3 (CLAUDE.md MANDATORY DEVELOPMENT PRINCIPLES) — *any method called from multiple threads must be safe for rapid consecutive calls* — is the contract this bank proves. The HelixVPN places where two callers race over shared state are concrete and high-stakes: a double-allocated overlay IP breaks routing for two devices; a lost policy edit grants access that was revoked; an out-of-order delta delivery leaves a device with a stale map. The bank covers the control plane's shared-state hot spots and the at-least-once event substrate.

Surface	Concurrent contention	Failure if unsafe	Invariant
IPAM allocator	N devices allocate overlay IPs simultaneously	two devices get the same IP	<code>overlay_pools.next_host</code> monotonic, SELECT ... FOR UPDATE [ipam §0.1]
device enrollment	the same device enrolls twice concurrently	duplicate identity / duplicate cert	unique constraint on (tenant_id, device_id)
policy edit	two admins edit the same tenant's policy	lost update (one edit silently dropped)	optimistic version guard on <code>policies.version</code> [svc-policy]
coordinator delta apply	deltas arrive out of order during a new enroll	a device's final map $\neq$ compiled target	order-invariant apply; final map == CompiledPolicy [reconcile §0.2]
event consume (Redis Streams)	a consumed message is redelivered (at-least-once)	a reconcile is applied twice	idempotent apply keyed by (SpecHash, Version) [reconcile §0.3]
connector site assignment	two connectors attach simultaneously	two connectors get the	<code>overlay_pools.next_site</code> monotonic [ipam §0.1]

Surface	Concurrent contention	Failure if unsafe	Invariant
		same 16-bit site id	

1. The three correctness properties

(A) No lost update / no double-allocation (atomicity at the DB).

Concurrent allocations and edits are serialized by the DB contract, not by app-level hope. IPAM allocates under `SELECT ... FOR UPDATE` on the `overlay_pools` row (or a unique constraint on `devices.overlay_ip`); `N` concurrent `Allocate` calls MUST yield **exactly** `N` distinct IPs and **zero** constraint violations [ipam §0.1]. Policy edits use an optimistic version guard: `UPDATE policies SET spec=..., version=version+1 WHERE id=$1 AND version=$expected`; a stale-version writer is **rejected**, never silently wins (no lost update) [svc-policy].

(B) Order-invariant convergence. The coordinator's final served map for a device MUST be invariant to the **arrival order** of deltas (a metamorphic relation, overview §3.2): whether the "new peer enrolled" delta arrives before or after the "policy edit" delta, the device converges to the same final map == its `CompiledPolicy.VisibleTo[self] ∩ AllowedIPs` [reconcile §0.2, svc-policy §3]. A property test asserts this over randomized delta interleavings.

(C) Idempotency under replay. The event substrate (Redis Streams, consumer group coordinator) is **at-least-once**: a message can be redelivered ([reconcile §0.3]). The `reconcile apply` MUST be idempotent — applying `policy.compiled{Version=N, SpecHash=H}` twice MUST produce the identical final state and emit **no duplicate** side effect (no second cert issued, no second WG peer added). Idempotency is keyed on `(SpecHash, Version)`: a replayed event whose `(SpecHash, Version)` matches the last-applied marker is a no-op. This is the property a redelivery-replay test proves.

2. Harness

Property	Harness
(A) no double-alloc / no lost update	Go test spawning <code>N</code> goroutines against real Postgres (booted via containers, overview §5.2); asserts exactly- <code>N</code> -distinct + zero constraint violations

Property	Harness
(B) order-invariant convergence	property test (gopter) over randomized delta interleavings; asserts final-map invariance
(C) idempotency under replay	inject a duplicate Redis Streams message (XADD the same payload, or NACK→redeliver); assert the apply is a no-op

All run with the Go race detector enabled (`go test -race`, overlaps [race-deadlock.md]) so a logical-concurrency test also surfaces a memory race. The bank runs under `make test CONC` stage and is dispatched by `helix_qa`. Infra is real, rootless Podman (§11.4.76/.161) — **no mocks** below the unit layer (§11.4.27): a mocked DB cannot prove the real `FOR UPDATE/unique-constraint` contract.

3. Fixtures — real DB, no mocks (§11.4.27)

Fixture	What	Why real
real Postgres via containers/pkg/boot	the actual control-plane DB with the real DDL (overlay_pools, devices, policies)	the <code>FOR UPDATE/unique-constraint/version-guard</code> contracts only exist on the real engine
real Redis Streams via containers	the actual event substrate, consumer group coordinator	at-least-once redelivery is a real-broker behaviour, not stubbable
tenant_seed.sql	a seeded tenant with a real /48 overlay pool	allocations must draw from a real pool with a real next_host
policy_v0.yaml + two divergent edits	two real conflicting policy specs	the lost-update test needs two real version-N edits

4. Captured evidence (§11.4.69)

Every PASS cites an artifact under `qa-results/conc/<run-id>/`:

Test	Artifact	Asserts
	<code>allocated_set.json</code>	

Test	Artifact	Asserts
CONC-IPAM-NO-DOUBLE		the N-set has exactly N distinct IPs
CONC-IPAM-NO-DOUBLE	pg_constraint_violations.txt	count == 0
CONC-POLICY-NO-LOST-UPDATE	version_trace.json	every accepted edit incremented version; the rejected stale edit is logged, not silently dropped
CONC-CONVERGENCE-ORDER-INVARIANT	final_maps.json	every interleaving yields the identical final map per device
CONC-IDEMPOTENT-REPLAY	replay_sideeffects.json	the replayed event produced zero new side effects (no 2nd cert / 2nd peer)
CONC-ENROLL-IDEMPOTENT	enroll_result.json	the same device enrolled twice ⇒ one identity, one cert

The captured *set* (the allocated-IP set, the final-map set) is the evidence — a CONC PASS that only asserts "no error" is a B2 absence-of-error bluff (overview §0); it **MUST** assert the *positive shape* (exactly N distinct, identical final map, zero new side effects).

5. Determinism (§11.4.50)

Concurrency tests are the canonical flake source, so determinism is enforced **mechanically**: `ab_run_n_times "conc-ipam" 10`
`run_ipam_concurrent` runs the contention test **N=10** (the cycle-validation count, because concurrency is high-risk per §11.4.132) against the same artifact MD5 + same DB image; the evidence-hash is over the verdict tuple (`distinct_ips == N: bool`, `constraint_violations == 0: bool`, `deadlocks == 0: bool`). All 10 runs **MUST** yield the identical tuple — a single divergent run (a double-allocation that appears once in 10) is **auto-FAIL** with no flake escape. There is no "it usually passes" path; intermittent

is mechanically forbidden (§11.4.50). The property tests (B) use a fixed gopher seed for reproducibility, with the seed recorded in the evidence so a failure is replayable.

6. Acceptance gate

Gate	Bar	Evidence	Bound feature
CONC-IPAM-NO-DOUBLE	N=64 concurrent allocs $\Rightarrow$ 64 distinct IPs, 0 violations	allocated_set.json	F-POLICY-RECONCILE / IPAM
CONC-POLICY-NO-LOST-UPDATE	concurrent edits $\Rightarrow$ no lost update; stale edit rejected	version_trace.json	F-POLICY-RECONCILE (AC5)
CONC-CONVERGENCE-ORDER-INVARIANT	randomized delta order $\Rightarrow$ identical final map	final_maps.json	F-POLICY-RECONCILE
CONC-IDEMPOTENT-REPLAY	redelivered event $\Rightarrow$ no-op, zero new side effect	replay_sideeffects.json	F-REVOKE (AC6) / reconcile
CONC-ENROLL-IDEMPOTENT	double enroll $\Rightarrow$ one identity, one cert	enroll_result.json	F-AUTHZ-REACH (AC2)

CONC cells appear in the coverage ledger for F-POLICY-RECONCILE and F-REVOKE (overview §6.3). Run with `-race` (RACE overlap). A double-allocation or lost update is a release blocker — it is a silent correctness defect that no functional test would catch.

7. The paired §1.1 mutation

MUTATION (paired §1.1, gate CM-IPAM-ATOMIC-ALLOC):

Drop the `SELECT ... FOR UPDATE` row lock (or the unique constraint on

devices.overlay_ip) from the IPAM allocate path.

EXPECTED: under N=64 concurrent Allocate calls a collision appears –

allocated_set.json shows < 64 distinct IPs OR
pg_constraint_violations.txt > 0 → CONC-IPAM-NO-DOUBLE

FAILs

→ mutation caught (within the N=10 determinism window).

RESTORE: re-instate FOR UPDATE / the constraint; re-run → GREEN x10.

A second mutation (CM-POLICY-VERSION-GUARD) removes the AND version=\$expected clause from the policy UPDATE; expected: a stale-version edit silently overwrites a newer one → version_trace.json shows a lost update → CONC-POLICY-NO-LOST-UPDATE FAILs → caught. A third (CM-RECONCILE-IDEMPOTENT) removes the (SpecHash, Version) last-applied marker check; expected: the replayed event issues a **second** cert / adds a **second** peer → CONC-IDEMPOTENT-REPLAY FAILs. All mutations restored, tree verified quiescent (§11.4.84) before commit.

8. Test skeletons

```
// helix-go/internal/ipam/conc_alloc_test.go – CONC-IPAM-NO-DOUBLE
// (overview §5.10)
//go:build integration
func TestIPAMNoDoubleAllocationUnderConcurrency(t *testing.T) {
    infra := mustBootPG(t) //
    containers submodule, rootless §11.4.161
    alloc := NewIPAMAllocator(infra.PostgresDSN(),
        mustParseULA("fd7a:115c:a1e0::/48"))
    const N = 64
    got := make([]netip.Addr, N); var wg sync.WaitGroup
    for i := 0; i < N; i++ { wg.Add(1); go func(i int){ defer
        wg.Done()
        a, err := alloc.Allocate(ctx, deviceID(i));
        requireNoErr(t, err); got[i] = a }(i) }
    wg.Wait()
    requireAllDistinct(t, got) // the N-
    set IS the captured evidence
    requireZeroConstraintViolations(t, infra)
    writeEvidence(t, "qa-results/conc/allocated_set.json",
        got) // §11.4.69
    // paired §1.1 (CM-IPAM-ATOMIC-ALLOC): drop FOR UPDATE → a
    collision appears → FAIL
}
```

```

// CONC-CONVERGENCE-ORDER-INVARIANT – property test over
// randomized delta interleavings (B)
func TestFinalMapInvariantToDeltaOrder(t *testing.T) {
    props :=
        gopter.NewProperties(gopter.DefaultTestParametersWithSeed(0xHELIX)) //
        recorded seed
    props.Property("final map == compiled target regardless of
        delta order", prop.ForAll(
        func(order []DeltaIndex) bool {
            m := applyDeltasInOrder(baseMap, deltas, order)
            return m.Equal(compiledTargetFor(self)) //
            metamorphic: order-invariant convergence
        }, genPermutationOfDeltas()))
    props.TestingRun(t) //
        failures replayable from the recorded seed
}

// CONC-IDEMPOTENT-REPLAY – redeliver a consumed event, assert no-
// op (C)
func TestReconcileIdempotentUnderRedelivery(t *testing.T) {
    infra := mustBootRedisPG(t)
    ev := PolicyCompiled{Version: 7, SpecHash: "h7"}
    applyReconcile(t, infra, ev) // first
        delivery: applies
    certsBefore, peersBefore := snapshotSideEffects(t, infra)
    redeliverSameMessage(t, infra, ev) // at-
        least-once redelivery
    certsAfter, peersAfter := snapshotSideEffects(t, infra)
    requireEqual(t, certsBefore, certsAfter) // ZERO
        new cert
    requireEqual(t, peersBefore, peersAfter) // ZERO
        new WG peer
    writeEvidence(t, "qa-results/conc/replay_sideeffects.json",
        diff(certsBefore, certsAfter))
    // paired §1.1 (CM-RECONCILE-IDEMPOTENT): drop the
        (SpecHash,Version) marker → 2nd side effect → FAIL
}

```

Honest boundary (§11.4.6). This bank proves logical correctness for the enumerated shared-state hot spots; it does **not** prove freedom from *memory-model* data races (that is [race-deadlock.md]) nor from faults outside the contention set (covered by §11.4.118 discovery). The order-invariant convergence property is proven over the gopter-generated permutation space, which is a sample of the interleaving space, not exhaustive — exhaustive interleaving of the Rust critical sections is the loom job in [race-deadlock.md]. The "at-least-once $\Rightarrow$ idempotent-apply" guarantee rests on the Redis Streams consumer-group semantics ([reconcile §0.3]) and is **UNVERIFIED** for the specific redelivery-timing until the replay test runs.

Sources verified

- [OVERVIEW] [../10-testing-acceptance-and-qa.md] — §3.2 (metamorphic relations), §5.10 (CONC strategy + IPAM skeleton), §5.2 (containers INT), §6 (ledger), §8 (determinism N=10/risk-order). (Read 2026-06-26.)
- [ipam] [../v03-control-plane/svc-ipam.md] — §0.1 overlay_pools.next_host/next_site monotonic counters, connector_sites, ipam-computes-registry-persists-in-same-tx. (Read 2026-06-26.)
- [reconcile] [../v03-control-plane/reconciliation-flow.md] — §0.2 signature flow, §0.3 events:* Redis Streams consumer group coordinator, helix_reconcile_seconds. (Read 2026-06-26.)
- [svc-policy] [../v03-control-plane/svc-policy.md] — §3 CompiledPolicy {Version, SpecHash, VisibleTo, AllowedIPs}, policy version semantics. (Read 2026-06-26.)
- [TM] [../v05-security/threat-model.md] — T-CP-T-1/T-CP-I-1 (RLS), T-COORD-I-1 (filtered map). (Cited via overview.)
- Constitution: §11.4.169, §11.4.27 (no-fakes / real DB), §11.4.69 (sink-side evidence / positive shape), §11.4.50 (determinism N=10), §11.4.132 (concurrency is high-risk), §11.4.84 (quiescence), §11.4.76/.161 (containers/rootless), §1.1 (paired mutation); CLAUDE.md principle 3 (safe for rapid consecutive calls).